

PHASE2_BLOCKERS_INVESTIGATION

Phase-2 Blockers Investigation — HelixAgent → HelixLLM endpoint (OQ1) + Cognee memory-write (OQ2)

Revision: 1 **Last modified:** 2026-07-06 **Maintainer:** CLI-Agent Fusion programme
(systematic-debugging investigation, §11.4.102)

Scope	Root-cause investigation (read-only) of the two open blockers to the Phase-2 HelixAgent → HelixLLM end-to-end proof, per docs/research/07.2026/00_master/PHASE2_SERVICE_STACK.md §7 (Q1, Q6).
Method	§11.4.102 systematic-debugging; FACTs only, cited to file:line + git commits + captured command output. No source flipped/edited (config flip + provider change are separate reviewed changes).
Branch	feature/helixllm-full-extension

OQ1 — HelixAgent → HelixLLM endpoint ambiguity

CORRECTION (2026-07-07, post Phase-2 e2e proof — supersedes the :18434/v1 value below). This investigation asserted the pin `HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434/v1`. The Phase-2 e2e proof (docs/qa/phase2_e2e_20260706/12_endpoint_finding.txt, commit 278df582) then **PROVED** this is **wrong for the effective request**: the provider **hardcodes** the `/v1/chat/completions` suffix (provider.go:45 chatEndpoint, appended at :195), so a `.../v1` base yields a double `/v1/v1/chat/completions` → **HTTP 404**. The correct pin is the **BASE**

HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434
(**no /v1**). The resolveEndpoint precedence finding below stays valid
(it returns the env value verbatim); only the value to set is corrected.
Raw curl still uses http://localhost:18434/v1/chat/
completions directly. Authoritative current value: RESUME.md LIVE-
SERVER gotcha + submodules/helix_llm/docs/
OPERATOR_GUIDE.md.

FACT findings

F1 — resolveEndpoint precedence chain (cited). submodules/helix_agent/
internal/llm/providers/helixllm/provider.go:59-70 implements
resolveEndpoint(explicit string) with 4-level precedence:

1. explicit (caller / provider-registry baseURL) — provider.go:60-62
2. HELIX_LLM_LOCAL_OPENAI_ENDPOINT (EnvLocalOpenAIEndpoint,
provider.go:41, :63-65) — the local plain-HTTP OpenAI-router seam
3. HELIX_LLM_ENDPOINT (EnvEndpoint, provider.go:40, :66-68) — general
override
4. DefaultEndpoint = "https://localhost:8443" (provider.go:29, :69)
— TLS gateway default

NewProvider applies it at construction: cfg.Endpoint =
resolveEndpoint(cfg.Endpoint) (provider.go:116).

F2 — the three candidate endpoints classified (FACT). - :8443 (TLS gateway) —
the code default (provider.go:29) AND the .env.example default
HELIX_LLM_ENDPOINT=http://localhost:8443 (submodules/
helix_agent/.env.example:88). Aspirational: it is the legacy default, not the
proven-live server (per master §7-Q1 / RESUME.md). - :18434/v1 (llama-server,
plain HTTP) — the **only proven-live** server today (helixllm-coder container Up;
master §7-Q1). Reached ONLY via an env var; not a code default. - :8100/v1
(router) — planned (plan P6-T1); not yet serving. Aspirational.

F3 — which env var + value pins the proven-live coder (FACT, cited). Because
HELIX_LLM_LOCAL_OPENAI_ENDPOINT sits ABOVE HELIX_LLM_ENDPOINT in
precedence (provider.go:63-68), setting
HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434/v1 makes
HelixAgent's HelixLLM provider resolve to the proven-live coder regardless of the
.env.example :8443 default. :8443 and :8100 are aspirational (not serving
today).

PROOF (captured this session)

Precedence-chain unit test (internal/llm/providers/helixllm/
provider_test.go:102-171, added by commit 28ae35ea "test(helixllm):
cover full precedence chain of resolveEndpoint"):

```
$ go test -v -run 'TestResolveEndpoint_PrecedenceChain|TestNewProvider_Def
--- PASS: TestNewProvider_DefaultEndpoint (0.00s)
--- PASS: TestResolveEndpoint_PrecedenceChain (0.00s)
    --- PASS: .../explicit_endpoint_wins_over_everything
    --- PASS: .../only_local_OpenAI_endpoint_set_is_used
    --- PASS: .../only_general_endpoint_set_is_used
    --- PASS: .../both_local_and_general_set:_local_OpenAI_endpoint_wins
    --- PASS: .../nothing_set_falls_back_to_DefaultEndpoint
ok      dev.helix.agent/internal/llm/providers/helixllm 0.004s
```

Targeted harness proving the EXACT :18434/v1 resolution (temporary test file,
run then removed — working tree left clean, read-only investigation preserved):

```
$ go test -v -run TestOQ1_Pin18434 ./internal/llm/providers/helixllm/
zz_oq1harness_test.go:7: resolved endpoint = "http://localhost:18434/v1"
--- PASS: TestOQ1_Pin18434 (0.00s)
ok      dev.helix.agent/internal/llm/providers/helixllm 0.003s
```

Harness set only HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://
localhost:18434/v1 (general env unset) and asserted resolveEndpoint("")
== "http://localhost:18434/v1". Confirmed. (Full HelixAgent binary run
against live infra was NOT attempted — unit-level resolveEndpoint proof is the
correct evidence layer per §11.4.3; a live generate is a separate step once the
endpoint is pinned.)

OQ1 RECOMMENDATION

Pin, for the Phase-2 proof:

HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434/v1

Rationale: it is the highest-precedence env seam (provider.go:63-65), overrides
the aspirational :8443 .env.example default without editing it, and points at the
ONLY proven-live server. Do NOT rely on HELIX_LLM_ENDPOINT (it is lower
precedence and its .env.example value is the non-serving :8443). :8100 is not
yet serving. Leave HELIX_LLM_TLS_SKIP_VERIFY irrelevant here (the coder is
plain HTTP). This is a .env-only change — no provider-code edit required.

OQ2 — Cognition integration disabled

FACT findings

F4 — COGNEE_BUG.md DOES exist (master §7-Q6 correction). Master §7-Q6 states the referenced COGNEE_BUG.md was “not found in submodules/helix_agent/”. That is INCORRECT: the file is at **submodules/helix_agent/docs/COGNEE_BUG.md** (committed 51f7ee92, 2026-01-30). §7-Q6 looked at the submodule root, not docs/.

F5 — WHEN/WHY it was disabled (git history, §11.4.124). - configs/development.yaml:317 cognition_integration: false was introduced ALREADY-DISABLED on **2025-12-14** (commit afd0865c, which first added the cognition: block + cognition_integration: false). It was never enabled in development; the comment cites COGNEE_BUG.md. - The bug doc explaining WHY landed 6 weeks later (51f7ee92, 2026-01-30). - configs/production.yaml:230 has cognition_integration: true (production differs from development).

F6 — the upstream bug is REAL (documented root cause). docs/COGNEE_BUG.md records: AttributeError: 'str' object has no attribute 'nodes' at /app/cognition/tasks/memify/extract_subgraph_chunks.py:9 inside the Cognition CONTAINER — the extract_subgraph_chunks task receives a list of strings but expects subgraph objects with .nodes, so /api/v1/search, /api/v1/add, /api/v1/memify hang (~30 s timeout). Health + auth endpoints work. Tested against ghcr.io/topoteretes/cognition:latest (Jan 2026). Status in doc: **BLOCKED — waiting for upstream fix**. This is a genuine upstream-code bug, not a HelixAgent defect.

F7 — two distinct “memory-write” paths — do NOT conflate them. - (a) Cognition CONTAINER path (runtime write today): internal/handlers/cognition_api_handler.go:116 AddMemory (POST /cognition/memory) → services/cognition_service.go:616 CognitionService.AddMemory → container /api/v1/add (multipart, primary) with /api/v1/memify best-effort non-blocking fallback (cognition_service.go:614-736). Gated by s.config.Enabled (cognition_service.go:617). This path writes into the Cognition container’s own store — it does NOT write a cognition_memories Postgres row. It is config-disabled in development AND blocked by F6. - The two-phase add-primary/memify-fallback design (commit e99e5e44, 2026-01-29) is one day OLDER than the bug doc — i.e. it was the attempted workaround, and the doc a day later still records /api/v1/add timing out. So “add is reliable” was aspirational against the buggy :latest image. - **(b) Postgres cognition_memories row (the Phase-2 §11.4.108 signature target):** internal/database/cognition_memory_repository.go is a REAL, fully-implemented pgx repository (446 lines, Create → INSERT INTO cognition_memories at :42-66, plus 19 other real query methods — NOT stubbed,

no TODO/simulate/placeholder). Schema exists (sql/schema/cognee_memories.sql, internal/database/migrations/001_initial_schema.sql). **BUT it is UNWIRED:** NewCogneeMemoryRepository is constructed ONLY in _test.go files — grep of all non-test .go for CogneeMemoryRepository returns only its own definition + a doc-comment mention (internal/database/doc.go:43). No service, handler, or server wiring instantiates it. So a real HelixAgent generate does NOT currently persist a cognee_memories row — nothing on the request path calls the repository.

FACT verdict — is the memory-write path (a) usable now / (b) after a config flip / (c) blocked?

NEITHER a simple config flip NOR usable now. Two independent gaps stack: 1. **Wiring gap (repository unwired):** the Phase-2 signature target — a cognee_memories Postgres row written by a real generate — cannot occur today because CogneeMemoryRepository has zero production callers (F7b). A config flip does not wire it. 2. **Upstream-bug gap (container path):** flipping cognee.enabled / cognee_integration: true enables the CONTAINER path (F7a), whose write endpoints are blocked by the real upstream bug F6, verified only against ghcr.io/topoteretes/cognee:latest (Jan 2026). Whether the pinned cognee/cognee:1.2.2 (2026-06-26, master §7-Q4) fixes the extract_subgraph_chunks AttributeError is **UNVERIFIED** — not tested this session, no live container exercised.

OQ2 RECOMMENDATION

The Phase-2 §11.4.108 memory-write signature (“a real generate persisting a cognee_memories row”) **MUST be an honest §11.4.3 SKIP with a tracked work item** for the current Phase-2 proof — it is NOT provable today, and NOT unblocked by a config flip alone. Two prerequisites (both separate reviewed changes) must land first:

- **P-OQ2-A (wiring):** wire CogneeMemoryRepository into the generate/memory request path so a real generate persists a cognee_memories row (currently the repo is dead-wired — investigate-before-remove per §11.4.124; it should be WIRED, not removed). This makes the Postgres-row signature reachable independently of the buggy container.
- **P-OQ2-B (upstream-bug re-verification):** re-verify the F6 extract_subgraph_chunks bug against the digest-pinned cognee/cognee:1.2.2 image (§7-Q4) with a real /api/v1/add + /api/v1/memify call capturing wire evidence, BEFORE re-enabling cognee_integration. Do NOT assume 1.2.2 fixes it (§11.4.6 no-guessing); the current evidence is a Jan-2026 :latest failure.

Honest boundary: if P-OQ2-A lands, the **Postgres-row** signature becomes provable WITHOUT the container (the repository is real and the table exists) — that is the cheaper path to a genuine §11.4.108 memory-write signature. The container graph-enrichment (search/memify) remains gated on P-OQ2-B. Recommend the Phase-2 memory-write signature target the wired Postgres-row path (P-OQ2-A), and track container enrichment (P-OQ2-B) as a separate item so the master §7-Q6 blocker is decomposed correctly.

Summary

Blocker	Verdict	Pin / action
OQ1 endpoint	RESOLVED (proven)	Set HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434/v1 (highest-precedence env seam, provider.go:63-65; captured PASS). :8443/:8100 aspirational. .env-only, no code edit.
OQ2 cognee	SKIP-with-tracked-item (real 2-part blocker)	Postgres cognee_memories repo is real but UNWIRED (P-OQ2-A); container path is config-disabled + blocked by a real upstream extract_subgraph_chunks AttributeError (F6), re-verify vs cognee/cognee:1.2.2 (P-OQ2-B). Not a config flip.

Master §7 corrections captured: **§7-Q6 was wrong** — COGNEE_BUG.md exists at submodules/helix_agent/docs/COGNEE_BUG.md; the upstream bug is real and documented (not merely a missing file), and the blocker is two-part (wiring + upstream), not a single config toggle.